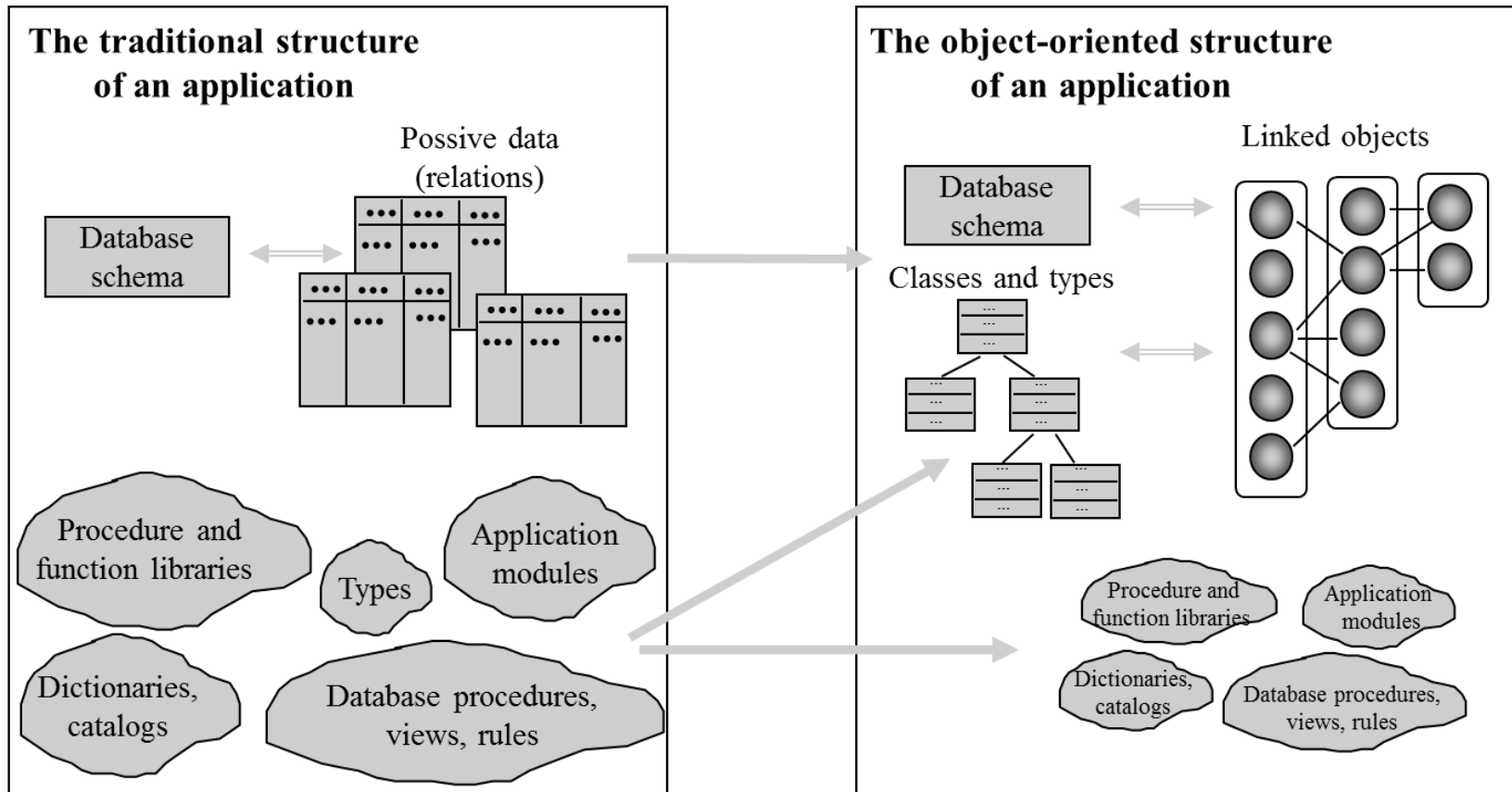


Pemrograman Berorientasi Objek



JDBC

Traditional vs object-oriented application



RDBMS vs OODBMS

- Relational databases store data in tables with two dimensional way.
 - The tables have rows and columns.
- Relational database tables are "normalized"
 - data is not repeated more often than necessary.
- All table columns depend on a primary key (a unique value in the column) to identify the column.
 - Once the specific column is identified, data from one or more rows associated with that column may be obtained or changed.

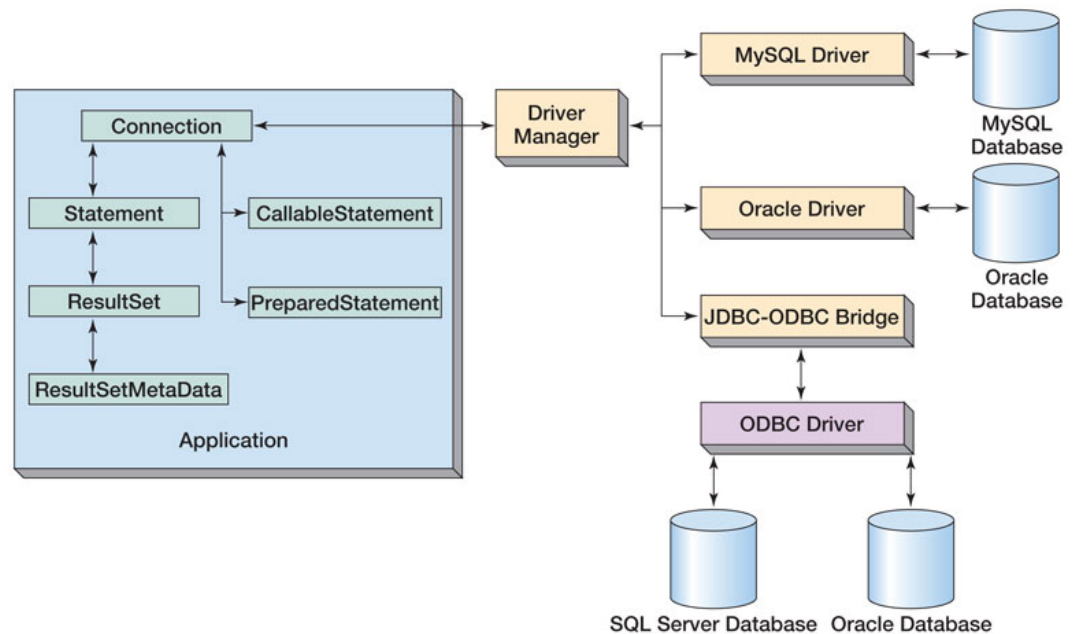
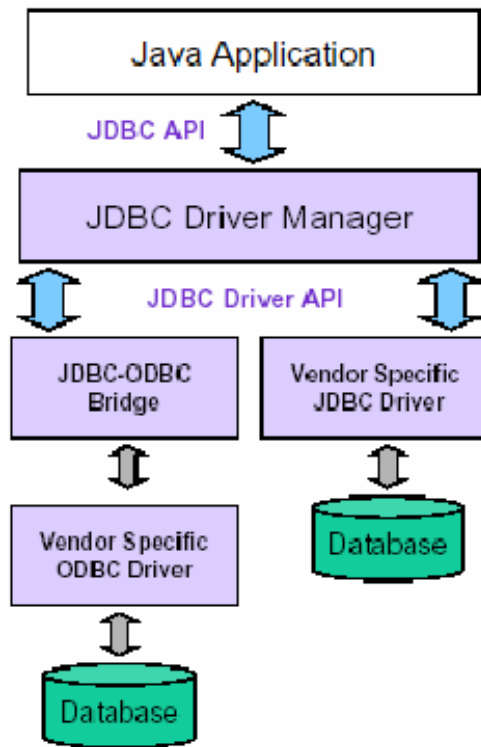
RDBMS vs OODBMS

- ▶ OO databases store data in tables with one dimensional way.
 - Rows (data) are converted into model objects
- ▶ OO database tables can be not-"normalized"
 - data can be repeated because the model are linked objects.
- ▶ All model objects depend on a key to identify the entire model object.
 - Model object may be obtained or changed.

Java Database Connectivity

- ▶ An API that lets you access virtually any tabular data source from the Java programming language
- ▶ an interface which allows Java code to execute SQL statements inside relational databases
 - the databases must follow the ANSI SQL-2 standard

General Architecture



Database Programming Steps

- Load the library
- Establish a connection
- Begin transaction (optional)
- Create a statement object
- Associate SQL with the statement object
- Provide values for statement parameters (optional)
- Execute the statement object
- Process the results
- End transaction (optional, if you use a transaction)
- Release resources

Using JDBC

► Load the library:

- The driver class libraries need to be in the CLASSPATH for the Java compiler and for the Java virtual machine.
- The most reliable way to load the driver into the program is:

- `Class.forName(string);`
- String is filled with the name of your DBMS-specific JDBC library
- Example:

```
Class.forName("com.mysql.jdbc.Driver");
```


Using JDBC

- ▶ Establish a connection to the database:
 - A connection URL string includes the literal jdbc:, followed by the name of the driver and a URL to the database
 - `String url = jdbc:oracle:thin:@reddwarf.cs.rit.edu:1521:csodb;`
 - or
 - `String url = jdbc:mysql://localhost:3306/db\_name;`
 - Create a Connection object:
`Connection conn = DriverManager.getConnection(url, dbUser, dbPassword)`

Using JDBC

▶ Begin the transaction

```
-conn.setTransactionIsolation(connection.TRANSACTION_SERIALIZABLE );  
-conn.setAutoCommit(false);
```

▶ Create a statement object

```
Statement stmt = conn.createStatement();
```

▶ Associate SQL with the statement object

```
String queryString = "create table students " +  
"(name varchar(30), id int, phone char(9))";
```

Using JDBC

► Process the statement:

- For query SELECT:

```
ResultSet rs = stmt.executeQuery(querystring);
```

- For query DDL & DML:

```
int result = stmt.executeUpdate(updatestring);
```

- Compiled queries can be processed via a PreparedStatement object
- Stored procedures can be processed via a CallableStatement object

Using JDBC

▶ End transaction

- `conn.commit();`
- `conn.rollback();`

▶ Release resources

- `stmt.close();`
- `con.close();`



Example

Prepare the Database

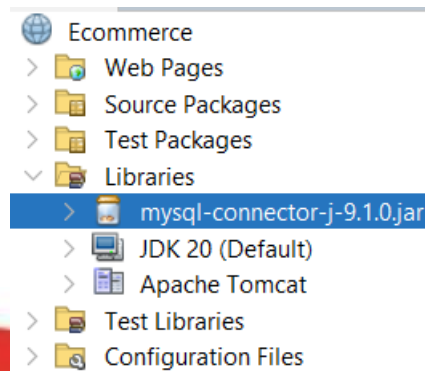
- ▶ We will use a RDBMS. Create a database directly from your DBMS or via DB Manager App
 - For example, if you use MySQL in XAMPP, the DB Manager App is phpMyAdmin (<http://localhost/phpmyadmin>)
- ▶ Schema:
 - DB name: exercise
 - Table: product
 - Field 1: id, int, PK, auto-increment
 - Field 2: name, varchar 50
 - Field 3: price, double

DB connection configuration

- ▶ Create a class for connection, add a procedure to connect (see Practical Work module)
- ▶ If you have different configuration, you can change the library string, DBMS server address & port, database name, or DBMS username & password

JDBC library

- ▶ If you develop the app with database, you must include the JDBC library in the project
- ▶ You can put the JDBC library anywhere, with relative path or absolute path (not recommended)
- ▶ This example project uses MySQL/MariaDB. The JDBC library is put relatively in 'lib' folder



Database actions

- Standard database actions (open/close connection, get, insert, update, delete) are the same for all model/entity, so the best practice is you create an abstraction for this, so you don't repeat creating all these actions in all of your model/controller class. This mechanism usually used in frameworks
- For starters, see the Practical Work module, where database actions is put in controller class
- In this slide we will do better practice: put the database actions in model class

From ResultSet to Model Class

- ▶ The result of your SELECT query is a ResultSet object. It's like a List of Object, but a bit complex
- ▶ Because we use RDBMS, we must manually define the column name or the index position to get the data from ResultSet object. After that, we will set the data to the class attributes
- ▶ You can use ResultSet directly (like in the example from Practical Work module) or create a function for this matter. This function will set the class attributes based on the column input in the ResultSet object

Mapping ResultSet to Model Class

- ▶ Open Product.java, add extends to JDBC connection class
- ▶ Add default constructor

```
public class Product extends JDBC {  
    private int id;  
    private String name;  
    private double price;  
  
    public Product() {  
  
    }  
}
```

Mapping ResultSet to Model Class

- Add function `toModel()` in `Product.java`

```
Product toModel(ResultSet rs) {  
    try {  
        return new Product(  
            rs.getInt("id"),  
            rs.getString("name"),  
            rs.getDouble("price")  
        );  
    } catch (Exception e) {  
        setMessage(e.getMessage());  
    }  
    return null;  
}
```

From ResultSet to ArrayList of Model

- Add function `get()` in `Product.java`

```
public ArrayList<Product> get() {  
    ArrayList<Product> res = new ArrayList<>();  
    try {  
        ResultSet rs = getData("SELECT * FROM product");  
        while (rs.next()) {  
            res.add(toModel(rs));  
        }  
    } catch (Exception e) {  
        setMessage(e.getMessage());  
    }  
    return res;  
}
```

MVC Implementation

- Your project must apply the MVC flow
- Best practice is the database actions are separated based on the HTTP method and request data (parameter) in the controller

View Data as ArrayList of Model

- Add this code in the Product controller function doGet () :

```
protected void doGet(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {
    String menu = request.getParameter("menu");
    if (request.getParameterMap().isEmpty()) { //view menu Model
        request.setAttribute("title", "Daftar Produk");
        ArrayList<Product> prods = new Product().get();
        request.setAttribute("list", prods);
        request.getRequestDispatcher("view_product.jsp").forward(request, response);
        request.getSession().removeAttribute("msg");
    }
}
```

view_product.jsp

- ▶ Use the view_product.jsp in the previous topic
- ▶ Try it:
localhost:8080/[your-project]/[product-servlet]

```
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<%@page import="java.util.ArrayList, models.Product" %>
<!DOCTYPE html>
<html>
  <head>
    <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
    <title>${title}</title>
  </head>
  <body>
    <h3>${title}</h3>
    <table border="1">
      <tr>
        <th>No</th>
        <th>Nama</th>
        <th>Harga</th>
      </tr>
      <%
        ArrayList<Product> prods = (ArrayList<Product>)request.getAttribute("list");
        int i = 1;
        for (Product p : prods) {
      <%
      <tr>
        <th><%= i++ %></th>
        <th><%= p.getName() %></th>
        <th><%= p.getPrice() %></th>
      </tr>
      <% } %>
      </table>
    </body>
  </html>
```


View Data as Plain Object

- ▶ This approach is useful when you run a complex or custom query like aggregation or subquery, which can produce columns that are not an attribute of a model. So, we will treat the result as a plain object and get the data by its column position in the query, not by the column name
- ▶ For example this query:

```
SELECT COUNT(*), AVG(price) FROM product
```

- COUNT(*) and AVG(price) aren't columns that associated with an attribute of a model, so you can use plain object instead of Model, and it's easier to get the data:
 - COUNT(*) : `obj_name.get(0)`
 - AVG(price) : `obj_name.get(1)`

Mapping ResultSet to Object

- Add function `toRow()` in `Product.java`

```
ArrayList<Object> toRow(ResultSet rs) {  
    ArrayList<Object> res = new ArrayList<>();  
    int i = 1;  
    try {  
        while (true) {  
            res.add(rs.getObject(i));  
            i++;  
        }  
    } catch (Exception e) {  
    }  
    return res;  
}
```

From ResultSet to ArrayList of Object

- Add function `query()` in `Product.java`

```
public ArrayList<ArrayList<Object>> query(String query) {  
    ArrayList<ArrayList<Object>> res = new ArrayList<>();  
    try {  
        ResultSet rs = getData(query);  
        while (rs.next()) {  
            res.add(toRow(rs));  
        }  
    } catch (Exception e) {  
        setMessage(e.getMessage());  
    }  
    return res;  
}
```

View Data as Plain Object

- Add this code in the Product controller function doGet () :

```
} else if ("custom".equals(menu)) { //view menu Object
    request.setAttribute("title", "Dashboard");
    ArrayList<ArrayList<Object>> prods = new Product().query(
        "SELECT COUNT(*), AVG(price) FROM product");
    request.setAttribute("list", prods);
    request.getRequestDispatcher("view_custom.jsp").forward(request, response);
}
```

view_custom.jsp

```
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<%@page import="java.util.ArrayList, models.Product"%>
<!DOCTYPE html>
<html>
  <head>
    <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
    <title>${title}</title>
  </head>
  <body>
    <h1>${title}</h1>
    <%
      ArrayList<ArrayList<Object>> prods = (ArrayList<ArrayList<Object>>)request.getAttribute("list");
      ArrayList<Object> p = prods.get(0);
    %>
    <h2>Jumlah produk : <%= p.get(0) %></h2>
    <h2>Rata-rata harga : <%= String.format("%.1f", p.get(1)) %></h2>
  </body>
</html>
```

- Try it: localhost:8080/[your-project]/[product-servlet]?menu=custom

Data Management

- Add the code for insert, update, delete, and find in Product.java

```
public Product find(String id) {
    ResultSet rs = getData("SELECT * FROM product WHERE id = " + id);
    try {
        if (rs.next()) {
            return toModel(rs);
        }
    } catch (Exception e) {
        setMessage(e.getMessage());
    }
    return null;
}

public void insert() {
    runQuery("INSERT INTO product(name, price)"
            + " VALUES ('" + name + "', '" + price + "')");
}

public void update() {
    runQuery("UPDATE product SET name = '" + name + "', price = '" + price + "'"
            + " WHERE id = " + id);
}

public void delete() {
    runQuery("DELETE FROM product WHERE id = " + id);
}
```

Add Data

- ▶ Adding data has two process: first is open the form page (GET method) and second is process the submitted data from the form page (POST method)
- ▶ For first process, add this in the Product controller function `doGet()`

```
} else if ("add".equals(menu)) { //add menu
    request.setAttribute("title", "Tambah Produk");
    request.getRequestDispatcher("form_product.jsp").forward(request, response);
}
```

Add Data

► Create JSP 'form_product'

```
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<%@page import="models.Product"%>
<!DOCTYPE html>
<html>
  <head>
    <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
    <title>${title}</title>
  </head>
  <body>
    <h4>Form ${title}</h4>
    <%
      Product prod = null;
      if (request.getAttribute("product") != null) prod = (Product)request.getAttribute("product");
    %>
    <form class="border" style="padding:20px" method="POST" action="product${action}">
      <div>
        <label>Nama</label>
        <input type="text" name="name" class="form-control" value="<%= prod == null ? "" : prod.getName() %>" required />
      </div>
      <div>
        <label>Harga</label>
        <input type="number" name="price" class="form-control" value="<%= prod == null ? "" : prod.getPrice() %>" required />
      </div>
      <div>
        <button class="btn btn-success">Simpan</button>
      </div>
    </form>
  </body>
</html>
```


Add Data

- ▶ For second process, add this in the function

doPost ()

```
protected void doPost(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {
    String action = request.getParameter("action");
    String id = request.getParameter("id");
    if (id == null) { //insert data
        Product p = new Product();
        p.setName(request.getParameter("name"));
        p.setPrice(Double.parseDouble(request.getParameter("price")));
        p.insert();
    }

    response.sendRedirect("product");
}
```

- ▶ Try it: localhost:8080/[your-project]/[product-servlet]?menu=add
- ▶ Click the add button, fill the form, submit

Edit Data

- ▶ Editing data has three process:
 - First: Load the data that corresponded with the id
 - Second: Open the form page and pre-fill the field with data attributes (GET method)
 - Third: process the submitted data from the form page (POST method)

Edit Data

- ▶ For first and second process, add this in the function `doGet()`

```
} else if ("edit".equals(menu)) { //edit menu
    request.setAttribute("title", "Edit Produk");
    request.setAttribute("action", "?id=" + request.getParameter("id"));
    Product p = new Product().find(request.getParameter("id"));
    if (p == null) {
        response.sendRedirect("product");
        return;
    }
    request.setAttribute("product", p);
    request.getRequestDispatcher("form_product.jsp").forward(request, response);
}
```

Edit Data

- ▶ For third process, add this in the function `doPost()`

```
} else if (action == null) { //update data
    Product p = new Product();
    p.setId(Integer.parseInt(id));
    p.setName(request.getParameter("name"));
    p.setPrice(Double.parseDouble(request.getParameter("price")));
    p.update();
    request.getSession().setAttribute("msg", p.getMessage());
}
```

- ▶ Try it: Click the edit button, fill the form, submit

Delete Data

- ▶ Deleting data just has one process: send the data id that wants to be deleted with POST method
- ▶ Add this in the function `doPost()`

```
} else if ("del".equals(action)) { //delete data
    Product p = (Product)new Product().find(id);
    if (p != null) {
        p.delete();
    }
}
```

- ▶ Try it: Click the delete button, confirms

Query Builder

- ▶ Sometimes you need more for database handling or more complex query features
- ▶ You can add your own function in the model, or use ORM library / framework

Hibernate ORM

- ▶ an object-relational mapping framework for the Java language, providing a framework for mapping an object-oriented domain model to a traditional relational database.
- ▶ Hibernate solves object-relational impedance mismatch problems by replacing direct persistence-related database accesses with high-level object handling functions.
- ▶ Explore it yourself 😊



Session

- ▶ Resource variable to store data globally
- ▶ Usually used for storing confidential or repetitive information (example: login status, account info, e-commerce cart, etc.)

Session

- ▶ Data (variable) is stored and managed by web server
- ▶ Time-out duration is taken from web server configuration
- ▶ Every client (browser) will be given different session id
- ▶ When client is closed, data is automatically erased

Session

▶ Storing & accessing session

//Store a session

```
request.getSession().setAttribute("username", username);
```

//Access a session in servlet

```
request.getSession().getAttribute("username")
```

<!-- Access a session in JSP -->

```
<%= request.getSession().getAttribute("username") %> or
```

```
<%= session.getAttribute("username") %> or
```

```
${sessionScope.username}
```

Session

▶ Deleting session

```
//delete specific session data
```

```
request.getSession().removeAttribute("username");
```

```
//delete all session data
```

```
request.getSession().invalidate();
```

Example with session: form_search.jsp

```
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
  <head>
    <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
    <title>Search Page</title>
  </head>
  <body>
    <h1>Hello, ${sessionScope.username}</h1>
    <a href="auth?logout"><button>Logout</button></a>
    <br /><br />
    <form method="get" action="home">
      Keyword : <input type="text" name="key" /><br />
      <input type="submit" value="Search" />
    </form>
    ${keyword}
  </body>
</html>
```

Example with session: Auth Servlet

```
@Override
protected void doGet(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {
    if (request.getParameter(name: "logout") != null)
        request.getSession().removeAttribute(name: "username");
    request.getRequestDispatcher(path: "form_login.jsp").forward(request, response);
}

@Override
protected void doPost(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {
    String username = request.getParameter(name: "user");
    String password = request.getParameter(name: "pass");
    if ("admin".equals(anObject: username) && "admin123".equals(anObject: password)) {
        request.getSession().setAttribute(name: "username", value: username);
        response.sendRedirect(location: "home");
        return;
    }
    request.setAttribute(name: "msg", o: "username / password salah");
    request.getRequestDispatcher(path: "form_login.jsp").forward(request, response);
}
```

Example with session: Home Servlet

```
@WebServlet(name = "Home", urlPatterns = {"/home"})
public class HomeController extends HttpServlet {

    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        if (request.getSession().getAttribute(name: "username") == null) {
            response.sendRedirect(location: "auth");
            return;
        }

        String keyword = request.getParameter(name: "key");
        request.setAttribute(name: "keyword", o: keyword);
        request.getRequestDispatcher(path: "form_search.jsp").forward(request, response);
    }
}
```

In the example with session before:

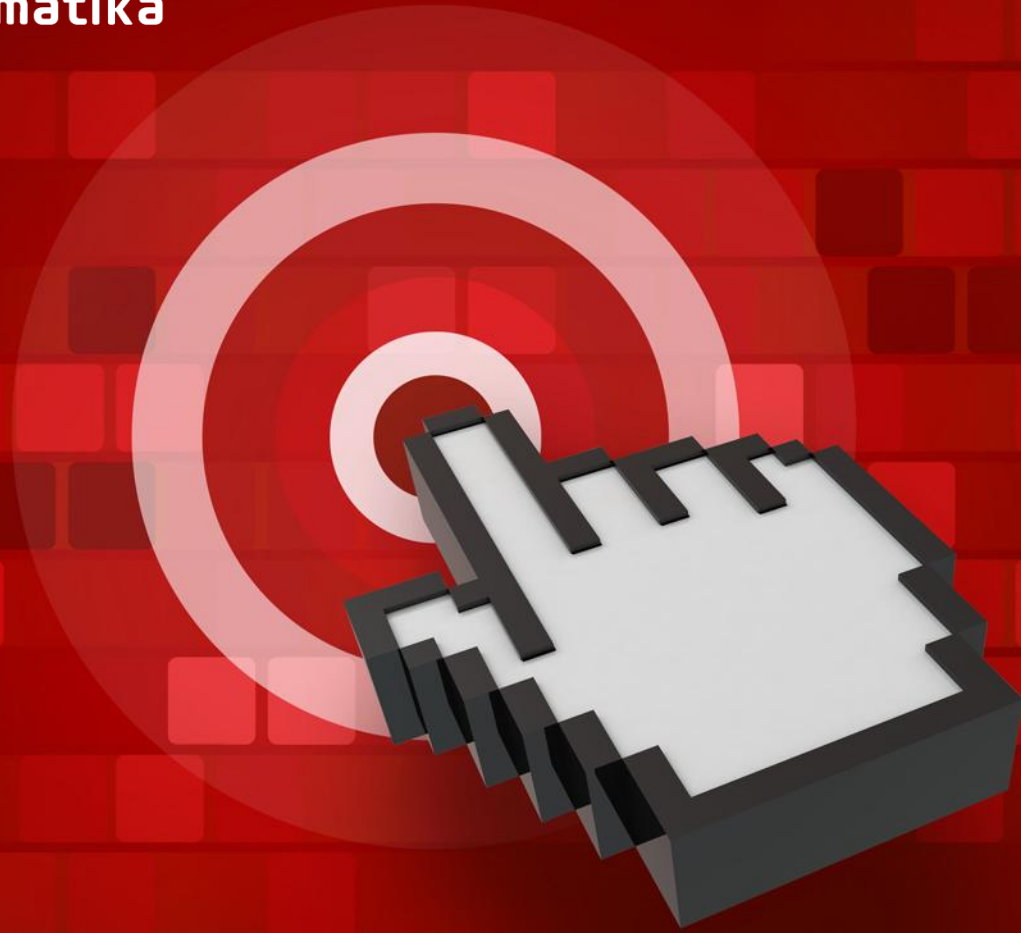
- ▶ You can store data that can be accessed across JSPs and servlets
- ▶ The session becomes the authentication, now you can't access '/home' address without login first
- ▶ The additional request data *logout* in '/auth' address is used to differentiate whether the action is 'just about to login' or 'just logged out'.

Question?





Fakultas Informatika
School of Computing
Telkom University



THANK YOU